Label propagation clustering

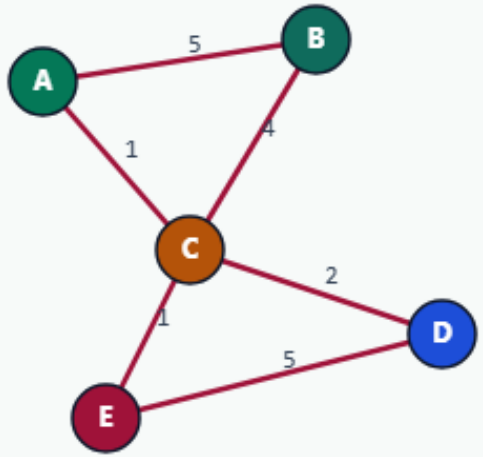
Label propagation grows communities by letting each node adopt the strongest neighbor label

Initialize: each node is its own label

LABEL PROPAGATION

Initialize: each node is its own label

Step 1 / 5 · init · module02-01-label-propagation



Explanation

Label propagation starts with five communities. Every node votes using weighted neighbor labels; async updates walk nodes in order A…E.

- labels[v] = v initially
- No global objective—local majority votes
- Order matters for ties

```
labels: A,B,C,D,E
num_clusters: 5
```

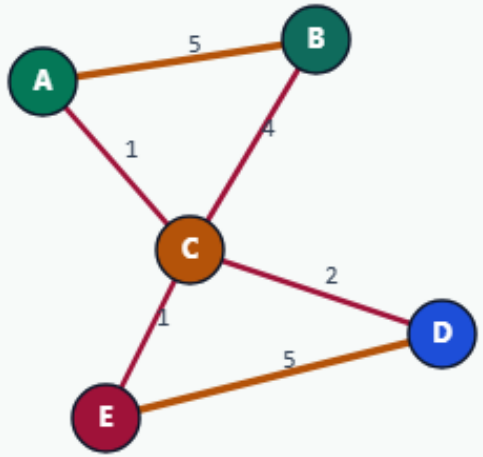
Initialize: each node is its own label

Each node adopts the winning neighbor label

LABEL PROPAGATION

Each node adopts the winning neighbor label

Step 2 / 5 · vote-idea · module02-01-label-propagation



Explanation

For a node v, sum edge weights by neighbor label and take the best. Dense A–B–C and D–E pull neighbors onto shared labels quickly.

- vote(label) = Σ w(v, nbr) for nbrs with that label
- Ties broken lexicographically
- One sweep = one iteration

```
Focus: A hears B strongly (w=5)
D hears E strongly (w=5)
```

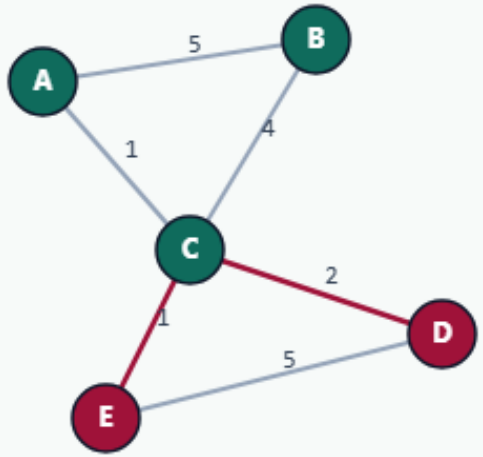
Each node adopts the winning neighbor label

After iteration 1: already clustered

LABEL PROPAGATION

After iteration 1: already clustered

Step 3 / 5 · after-iter1 · module02-01-label-propagation



Explanation

One async sweep flips A and C onto B, and D onto E. Communities {A,B,C} and {D,E} appear immediately on this tiny graph.

- 3 labels changed in iter 1
- A,B,C → label B
- D,E → label E

```
labels: {"A":"B","B":"B","C":"B","D":"E","E":"E"}
cutsize: 3
```

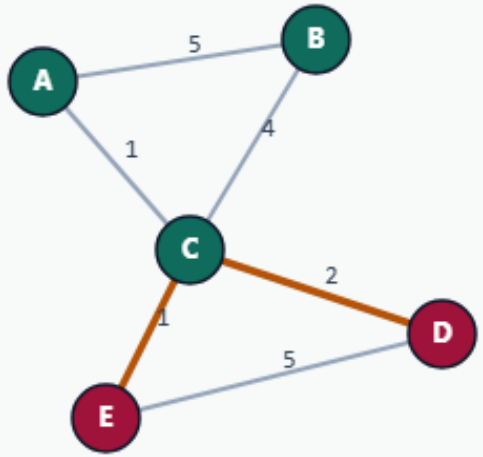
After iteration 1: already clustered

Iteration 2: no changes — stop

LABEL PROPAGATION

Iteration 2: no changes → stop

Step 4 / 5 · iter2-stable · module02-01-label-propagation



Explanation

A second sweep finds zero flips, so the algorithm reports iters_to_stable=2. Stability, not a cutsize objective, is the stopping rule.

- changed == 0 ⇒ halt
- Golden: iters=2, cutsize=3
- Same communities greedy found

```
iters_to_stable: 2
num_clusters: 2
cutsize: 3
```

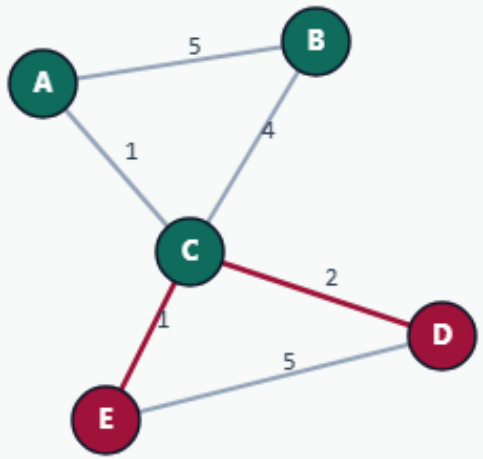
Iteration 2: no changes — stop

When LP helps in EDA flows

LABEL PROPAGATION

When LP helps in EDA flows

Step 5 / 5 · takeaway · module02-01-label-propagation



Explanation

LP is a cheap community detector—great as a seed or coarsening hint. It does not enforce balance or timing; refinement (KL/FM) still matters on hard instances.

- Fast local updates
- Sensitive to node order
- Use with capacity / cut objectives downstream

```
Reference communities: ABC | DE
```

When LP helps in EDA flows

Browser lab track

- In the browser lab, show the initial labels, then run label propagation
- Clear the challenges for two iterations, two communities, and the full golden label map

Implement track

- Load the tiny graph and run the reference label-propagation mode
- Confirm two iterations, labels grouping A–B–C versus D–E, and cutsize three
- Re-implement the vote and tie-break until the unit test passes

Implement track — try these

```
# run async label propagation on the tiny graph
export PYTHONPATH=../common

python ../common/solvers.py examples/tiny_graph.json --mode lp
```

Pitfalls to watch

- Order dependence is real, document your sweep order
- Without a tie-break, goldens flake
- And stopping only on max iterations without checking that nothing changed hides

Your turn

- Match the golden table, finish the checklist and quiz, then continue to spectral bisection